

/enhanced log file from SIMH that tells the tale. lower-case added within the word-processor to explain things.

.ZERO DTA1:

cleared the DECTape file.

.COPY DTA1:<RKB2:SMSORT.FC
FILES COPIED:
SMSORT.FC

copied the test .FC file to the first position in DTA1: That way the file is in record 7, in P?S/8 terms block 16 and perhaps 17 [turns out it was both].

.DIR DTA1: /E /B

15-JAN-79

SMSORT.FC 0007 1 15-OCT-78 <EMPTY> 0010 729

729 FREE BLOCKS

boot from OS/8 in SIMH to P?S/8 in SIMH. works fine by design.

.BO/DT

.BLKODT

1:16.\

In BLKODT terms, the : command has to have 0-7 in front of it, default is 0, but this needs to be logical drive unit 1.

the . command has to have a 12-bit block number in front of it, thus is block 16.

The \ command dumps out an entire block. The 0 command determines what is on the right. 0 is don't, 1 is standard sixbit ASCII [the default]. Values are defined up to around 7 for such as 7-bit ASCII and 3-for-2 packed 7-bit ASCII as used in the P?S/8 SHELL and OS/8 text files [and some others]. There are some other commands to change how much dumps out per \ command, but the default is one block per.

```
1:0016.000 0476 5051 0224 0000 0340 2557 2755 0617 *D>()BT@@C U/W-FO*
1:0016.010 0301 1472 0000 2315 2317 2224 0000 6165 *CAL:@@SMSORT@@15*
1:0016.020 5661 6056 6770 7715 0266 0212 2431 2005 *.10.78?MB6BJTYPE*
1:0016.030 4072 6161 4224 1011 2340 1123 4001 4004 * :11"THIS IS A D*
1:0016.040 0515 1716 2324 2201 2411 1716 4023 1005 *EMONSTRATION SHE*
1:0016.050 1414 5715 0524 3216 0522 4023 1722 2440 *LL/METZNER SORT *
1:0016.060 2022 1707 2201 1542 4177 1577 0330 0224 *PROGRAM"!M?CXBT*
1:0016.070 3205 2217 5411 5412 5413 5414 5415 7306 *ZERO,I,J,K,L,M;F*
1:0016.100 1722 4011 7561 5416 7562 6073 4023 0524 *OR I=1,N=20; SET*
1:0016.110 4001 5011 5175 6160 6052 0622 0116 5051 * A(I)=100*FRAN()*
1:0016.120 7340 1605 3024 7340 0417 4056 6577 1577 *; NEXT; DO .5?M?*
1:0016.130 0372 0236 0222 0501 1373 4016 0530 2473 *C:B^BREAK; NEXT;*
1:0016.140 4011 0640 5015 7506 1124 2250 1557 6251 * IF (M=FITR(M/2)*
1:0016.150 5154 5665 7340 0617 2240 1175 6154 1655 *),.5; FOR I=1,N-*
1:0016.160 1573 4004 1740 5664 7340 1605 3024 4056 *M; DO .4; NEXT .*
1:0016.170 6377 1577 0434 0250 0617 2240 1275 1154 *3?M?D\B(FOR J=I,*
```

17.\

Now go to block 17, all else stays what it was.

```
1:0017.000 5515 5461 7311 0640 5001 5012 5155 0150 *-M,1;IF (A(J)-A(*
1:0017.010 1375 1253 1551 5156 6354 5663 7340 2340 *K=J+M)).3,.3; S *
1:0017.020 0150 1251 7501 5013 5153 5652 0150 1351 *A(J)=A(K)+.*A(K)*
1:0017.030 7501 5012 5177 1577 0000 0262 2431 2005 *=A(J)?M?@@B2TYPE*
1:0017.040 4041 4564 5660 6273 0617 2240 1175 6154 * !%4.02;FOR I=1,*
1:0017.050 1657 6160 7306 1722 4013 7561 5461 6073 *N/10;FOR K=1,10;*
1:0017.060 2431 2005 4001 5015 7515 5361 5173 1605 *TYPE A(M=M+1);NE*
1:0017.070 3024 7324 3120 0540 4177 1577 4177 1440 *XT;TYPE !?M?!?L *
1:0017.100 0440 2315 2317 2224 7314 4023 4023 1523 *D SMSORT;L S SMS*
1:0017.110 1722 2473 0577 1577 0150 5462 6771 5140 *ORT;E?M?A(,279) *
1:0017.120 0622 0150 6154 6664 6351 4006 2201 5062 *FRA(1,643) FRA(2*
1:0017.130 5455 6151 4006 2201 5063 5455 0414 5361 *,-1) FRA(3,-DL+1*
1:0017.140 5140 0622 0150 7054 2302 5177 1577 0612 *) FRA(8,SB)?M?FJ*
1:0017.150 0320 2340 0622 0150 6454 5561 5140 0622 *CPS FRA(4,-1) FR*
1:0017.160 0150 6554 5561 5140 0622 0150 6654 6051 *A(5,-1) FRA(6,0)*
1:0017.170 4006 2201 5067 5406 2307 1650 6755 2302 * FRA(7,FSGN(7-SB)*
```

That is the entire file in this case.

^C

Control-C abort out of BLKODT

I pressed Control-E to get the rest.

```
.I:\Users\cjl\AppData\Local\Temp\SIMHCONFIG.INI-195> bo rk0
```

Simulation stopped, PC: 02344 (CLA)

I typed the "exit" below.

```
simh with TC08> exit
Goodbye
Log file closed
```

OK, that's the first pass at it.

Now to get some of the details.

For a typical line, the first word is the forward pointer to the address of the next line. The last forward pointer contains 0000 on the last line. Junk typically follows and should always be ignored.

The second word is the line number word.

Note: All FOCAL line numbers are stored as follows:

Bits[0-4] group number.

Bits[5-11] the local line number in the group.

The text buffer starts at 0200 [in whatever field this was created in]. This is apparent because the pointer words are consistent with this. Moreover, it just makes good sense to reserve page zero of any memory field.

Case-code ASCII is used. 77 is the reserved case flip, thus ? and the _ character are not allowed. Back then, the latter was the ← character.

All lines must be terminated with the expansion of <CR> which is 77 15. If there is an extra halfword, it is filled with 77 [which is ignored]. When flipped, you get lower-case printable.

Line breakdowns (Note: researched backwards from the end because it was easier for me that way, and some things realized along the way.)

```
1:0016.000 0476 5051 0224 0000 0340 2557 2755 0617 *D>()BT@@C U/W-FO*
1:0016.010 0301 1472 0000 2315 2317 2224 0000 6165 *CAL:@@SMSORT@@15*
1:0016.020 5661 6056 6770 7715 *.10.78?M *
```

I don't know the significance of the 0476 at relative 000 of the file. Somewhere I read about the 5051 being a U/W FOCAL-ism, some internal signature I guess because the program evolved and it has to do with capabilities as what they did to it parts company with the standard FOCAL language, which obviously happened (and not necessarily welcome). This is right out of the MIT playbook where students were never happy with standards, so they just broke them, thus the "TECO of the week" so show an analogous situation. Some of it is clearly chipping away at the language by allowing quirky syntax "JUST BECAUSE" but some other things are making quirky variants in the language (which really works well when you want to teach something and no two documents are alike because they cannot agree in what the subject matter is).

The rest is standard FOCAL innards. The first word is the forward pointer 0224 since the text buffer is somehow implied as going into x0200 where x is the field they used. Totally made up happenstance, so now they are stuck with it working that way. I see everything about this project as sloppy and memory wasteful in a time when it was a BIG DEAL to want another 4K. Seems they don't understand the notion of get something back for having paid for the hardware. This is a rather slim justification for all that memory usage considering the then-current cost. Overall, I am underwhelmed.

You have to read what follows in reverse order, so this is really my conclusion. But yes, you can do some minimal validation, and if it flunks, then who knows, or if half-assed validation works, perhaps one of the other programs also thinks of itself as *the* definitive FOCAL. That can be part of your program later, etc.

This can be displayed as:

```
C U/W-FOCAL: @@SMSORT@@15.10.78
```

There is no line number which is a standard part of the "badge" or "emblem" part of the buffer that is permanent before typing in even the first line. All FOCAL programs do that. Apparently some internal quirkiness to allow an internal functional title based on the non-extension filename. And then the date text. Notice DD/MM/YY which is very European and unexpected and arbitrary considering where this was written (perhaps just an arrogant conceit). Perhaps there is a way to assemble the program for alternate date format. Certainly not welcome in OS/8 which never handles dates that way, etc. If you want to bother, clearly the 0000 words are flags that the next that follows is the file name (no .FC; it apparently is required thus implicit in their little world) and then the text of the date of the file, thus you could turn it around with impunity into mm/dd/yy and because this is OS/8 you could add the leading 19xx on the year; it wouldn't be wrong.

I wrote P?S/8 date handling inherently Y2K-proof from the beginning. There is a secondary issue few have dealt with: If you allow two-digit years (which the P?S/8 DATE command does) you have to define the "favored century" the two-digit form applies to. Today, perhaps people want 2010 forward to 2050 and maybe back from 1980 to 1999. Thus, the code, written in the 1970s was actually thinking quite ahead.

In any case, in P?S/8, either will do for now:

```
.DATE 18-SEP-2017
or
.DATE 18-SEP-17
```

Line breakdowns with FOCAL line numbers.

```

1:0016.020                0266 0212 2431 2005 *          B6BJTYPE*
1:0016.030 4072 6161 4224 1011 2340 1123 4001 4004 * :11"THIS IS A D*
1:0016.040 0515 1716 2324 2201 2411 1716 4023 1005 *EMONSTRATION SHE*
1:0016.050 1414 5715 0524 3216 0522 4023 1722 2440 *LL/METZNER SORT *
1:0016.060 2022 1707 2201 1542 4177 1577      *PROGRAM"?M? *
```

This can be displayed as:

2.10 TYPE :11"THIS IS A DEMONSTRATION SHELL/METZNER SORT PROGRAM"!

```

1:0016.060                0330 0224 *          CXBT*
1:0016.070 3205 2217 5411 5412 5413 5414 5415 7306 *ZERO,I,J,K,L,M;F*
1:0016.100 1722 4011 7561 5416 7562 6073 4023 0524 *OR I=1,N=20; SET*
1:0016.110 4001 5011 5175 6160 6052 0622 0116 5051 * A(I)=100*FRAN()*
1:0016.120 7340 1605 3024 7340 0417 4056 6577 1577 *; NEXT; DO .5?M?*
```

This can be displayed as:

2.20 ZERO,I,J,K,L,M;FOR I=1,N,N=20; SET A(I)=100*FRAN(); NEXT; DO .5

```

1:0016.130 0372 0236 0222 0501 1373 4016 0530 2473 *C:B^BREAK; NEXT;*
1:0016.140 4011 0640 5015 7506 1124 2250 1557 6251 * IF (M=FITR(M/2)*
1:0016.150 5154 5665 7340 0617 2240 1175 6154 1655 *),.5; FOR I=1,N-*
1:0016.160 1573 4004 1740 5664 7340 1605 3024 4056 *M; DO .4; NEXT .*
1:0016.170 6377 1577      *3?M? *
```

This can be displayed as:

2.30 BREAK; NEXT; IF (M=FITR(M/2)),.5; FOR I=1,N-M; DO .4; NEXT .3

Note: I am reconstructing these backwards for convenience as I do them manually. Some odd language features have been added apparently!

```

1:0016.170                0434 0250 0617 2240 1275 1154 *      D\B(FOR J=I,*
1:0017.000 5515 5461 7311 0640 5001 5012 5155 0150 *-M,1;IF (A(J)-A(*
1:0017.010 1375 1253 1551 5156 6354 5663 7340 2340 *K=J+M)).3,.3; S *
1:0017.020 0150 1251 7501 5013 5153 5652 0150 1351 *A(J)=A(K)+.*A(K)*
1:0017.030 7501 5012 5177 1577      *=A(J)?M? *
```

This can be displayed as:

2.40 FOR J=I,-M,1;IF (A(J)-A(K=J+M)).3,.3; S A(J)=A(K)+.*A(K)=A(J)

The first word is the pointer to the next absolute address which is where line 2.50 stuff is (which is immediately following).

The first word is the pointer to the next absolute address; this one is 0000 indicating this is the last line. FOCAL needs to know where in memory the next free word is [which has old junk in it]. This line loads into 0434 in whatever field it was created in.

```

1:0017.030                0000 0262 2431 2005 *          @@B2TYPE*
1:0017.040 4041 4564 5660 6273 0617 2240 1175 6154 * !%4.02;FOR I=1,*
1:0017.050 1657 6160 7306 1722 4013 7561 5461 6073 *N/10;FOR K=1,10;*
1:0017.060 2431 2005 4001 5015 7515 5361 5173 1605 *TYPE A(M=M+1);NE*
1:0017.070 3024 7324 3120 0540 4177 1577      *XT;TYPE !?M? *
```

This can be printed as:

2.50 TYPE !%4.02;FOR I=1,N/10;FOR K=1,10;TYPE A(M=M+1);NEXT;TYPE !

This is the last line because the forward pointer address is 0000. Because the line length is odd, an extra ? is used as an ignored filler after the <CR> ends the line.

The rest of the file is junk. I removed the originals of the junk.

Ask away if you need more help, but I can see this as solved. If a .FC file, validate the few things (such as the 5051 near the beginning), and the chain of pointer addresses leading to a 0000 in the last one and the badge data has no line number as the unique case. I would even say we're safe with checking for U/W-FOCAL.

cjl